ne-body: System Architecture Document

Version 0.1 — Proof of Concept (POC) Project: Continuous Monitoring & Prediction Platform for Space Situational Awareness (SSA) **Status:** Draft — for funding review

---

1. Purpose and scope

This document describes the architecture of the ne-body proof-of-concept system. It covers the data flow from Two-Line Element (TLE) ingestion through orbital propagation, Kalman filter state estimation, anomaly detection, and browser-based visualization.

The POC scope is intentionally constrained: a single-node Python backend, a Content Delivery Network (CDN)-served browser frontend, and Space-Track.org as the sole data source. The architecture is designed so that each component can be replaced or scaled independently as the platform matures toward production.

---

2. Core architectural concept

Traditional SSA systems compute a trajectory once and extrapolate forward. Prediction error grows with time because orbital dynamics are chaotic — small unmodeled perturbations (atmospheric drag variations, solar radiation pressure, unannounced maneuvers) compound exponentially. The standard response has been to improve propagator fidelity, which has diminishing returns.

The ne-body approach abandons the long-horizon prediction model entirely. Instead of asking "how accurately can we predict position in 72 hours?", it asks "how quickly can we detect that our current prediction is wrong and correct it?" This reframes the problem from a mathematical accuracy challenge to a control systems problem — specifically, a recursive state estimation problem that Kalman filtering is well suited for.

The fundamental loop is:

```
TLE observation arrives
      │
      ▼
Propagate current state estimate forward to observation epoch
      │
      ▼
Compute residual: observed state − predicted state
      │
      ▼
Is residual within expected noise bounds (NIS test)?
  ├── YES → Update filter state; confidence remains high
  └── NO  → Anomaly flag; trigger recalibration; alert
      │
      ▼
Updated state estimate → propagate to next expected observation
```

Every new TLE update is both a data point and a validation of the previous prediction cycle. The system is always either confirming its model or correcting it. The Normalized Innovation Squared (NIS) test referenced above is defined formally in section 3.3.

---

3. System components

3.1 TLE ingest service (`backend/ingest.py`)

Purpose: Sole interface to Space-Track.org. Polls the Application Programming Interface (API) on a schedule, validates response structure, caches locally.

Behavior:

- Polls every 30 minutes (Space-Track rate limit compliant)
- Fetches TLEs for a curated catalog of 20–50 objects
- Validates TLE checksum and epoch recency
- Writes to local SQLite catalog table: `(norad_id, epoch_utc, tle_line1, tle_line2, fetched_at)`
- Emits a catalog update event on the internal event bus when new TLEs arrive
- All Space-Track credentials stored in environment variables, never in source

What it does not do: No propagation, no filtering, no API exposure. Its only job is reliable data acquisition and caching.

3.2 Propagation engine (`backend/propagator.py`)

Purpose: Convert TLE elements into Earth-Centered Inertial (ECI) state vectors at arbitrary epochs.

Implementation: Python `sgp4` library (Vallado implementation). For POC objects in low Earth orbit, SGP4 is sufficient. The module interface accepts a TLE and a target epoch and returns position and velocity in ECI kilometers and km/s.

Design note: The propagator is stateless. It takes inputs and returns a state vector; it holds no memory between calls. This makes it trivially testable and replaceable. A higher-fidelity numerical integrator (e.g., RK4 with J2/J3/drag force models) can be substituted at this interface with no changes to the rest of the system.

Coordinate frame: All outputs are ECI J2000. The propagator never outputs Earth-Centered Earth-Fixed (ECEF) or geodetic coordinates — conversion is the responsibility of the API layer.

3.3 Kalman filter engine (`backend/kalman.py`)

Purpose: Maintain a probabilistic state estimate for each tracked object across time. This is the architectural heart of the platform.

Implementation: Unscented Kalman Filter (UKF) via FilterPy. The UKF is chosen over the simpler Extended Kalman Filter (EKF) because the SGP4 measurement model is nonlinear and the EKF's first-order

linearization introduces systematic error that accumulates over time.

State vector: `[x, y, z, vx, vy, vz]` — position and velocity in ECI, 6-dimensional.

Process model: SGP4 propagation from one observation epoch to the next. The process noise matrix `Q` represents unmodeled forces (drag uncertainty, solar radiation pressure (SRP), maneuver probability). For POC, `Q` is hand-tuned per object class (debris vs. active satellite).

Measurement model: TLE-derived state vector is treated as a noisy observation. Measurement noise matrix `R` reflects TLE accuracy class (typically 100m–1km position uncertainty for publicly available TLEs).

Filter update cycle:

1. New TLE arrives for object with North American Aerospace Defense Command (NORAD) ID `n`
2. Propagate current state estimate from last update epoch to new TLE epoch (predict step)
3. Convert TLE to ECI state vector via propagator
4. Compute innovation (residual): `y = z_observed - z_predicted`
5. Compute NIS: `NIS = y^T * S^{-1} * y` where S is innovation covariance
6. If NIS exceeds chi-squared threshold (p=0.05, 6 degrees of freedom (dof)): flag anomaly, trigger recalibration
7. Execute UKF update step; store updated state and covariance
8. Emit state update event to API layer

Recalibration: When anomaly is flagged, the filter re-initializes from the new observation with inflated covariance, rather than attempting to update from a highly inconsistent prior.

3.4 Anomaly detection (`backend/anomaly.py`)

Purpose: Interpret filter residuals as events with operational meaning.

Detection logic:

- **Maneuver detection:** Sustained NIS elevation (>3 consecutive update cycles above threshold) on an active satellite catalog entry
- **Conjunction precursor:** Two objects' uncertainty ellipsoids overlap within a configurable threshold (default: 1 km Pc-equivalent)
- **Drag anomaly:** Systematic along-track residual growth without cross-track signature (indicates atmospheric density model error or unmodeled drag event)
- **Debris generation event:** Sudden increase in catalog objects in a common orbital regime (post-POC feature — flagged in requirements)

Output: Anomaly events are written to a `alerts` table in SQLite and pushed to connected WebSocket clients immediately.

3.5 API and WebSocket gateway (`backend/main.py`)

Purpose: Expose backend state to the browser frontend.

Endpoints:

- `GET /catalog` — list of tracked objects with current state summary
- `GET /object/{norad_id}/history` — state history for a single object
- `WebSocket /ws/live` — streaming channel: pushes state updates and anomaly alerts in real time

Message format (WebSocket):

```
{
  "type": "state_update" | "anomaly" | "recalibration",
  "norad_id": 25544,
  "epoch_utc": "2026-03-28T19:00:00Z",
  "eci_km": [x, y, z],
  "eci_km_s": [vx, vy, vz],
  "covariance_diagonal_km2": [σx², σy², σz²],
  "nis": 2.3,
  "innovation_eci_km": [dx, dy, dz, dvx, dvy, dvz],
  "confidence": 0.94,
  "anomaly_type": null | "maneuver" | "drag_anomaly" | "filter_divergence"
}
```

3.6 Browser frontend

3.6.1 3D orbital view (`frontend/src/globe.js`)

Library: CesiumJS (CDN, free lon tier for POC)

Rendered elements:

- Earth globe with realistic texture
- Satellite ground tracks (polyline, fading historical trail)
- Current position markers (billboard icons, color-coded by confidence level)
- Uncertainty ellipsoids: rendered as translucent ellipsoid entities scaled to 3σ from covariance diagonal
- Anomaly highlight: object flashes and ellipsoid turns amber when anomaly fires

CZML update strategy: The frontend maintains a Cesium Language (CZML) data source that is patched with each incoming WebSocket state update. CesiumJS handles interpolation between updates.

3.6.2 Residual timeline (`frontend/src/residuals.js`)

Library: D3.js v7 (CDN)

Charts:

- Per-object time series: residual magnitude vs. time, with ±2σ expected noise band
- NIS score over time, horizontal threshold line at chi-squared critical value
- Confidence score strip chart (0–1, colored green→amber→red)

Interaction: Clicking an object on the globe cross-filters the residual charts to that object.

3.6.3 Anomaly alert panel (`frontend/src/alerts.js`)

Behavior: Receives anomaly WebSocket events; renders a scrolling feed with:

- Timestamp, NORAD ID, object name, anomaly type
- Residual magnitude at detection
- Status: active / recalibrating / resolved
- Time-to-resolution once recalibration completes

---

4. Data flow diagram

The diagram below uses Hypertext Transfer Protocol Secure (HTTPS) to Space-Track.org and a JavaScript Object Notation (JSON) stream to the browser.

```
Space-Track.org
      │  (HTTPS, every 30 min)
      ▼
┌─────────────────┐
│  ingest.py      │──→ SQLite catalog
└─────────────────┘
      │ new TLE event
      ▼
┌─────────────────┐      ┌──────────────────┐
│  propagator.py  │◄─────│   kalman.py      │
│  (SGP4, ECI)    │      │  (UKF per object) │
└─────────────────┘      └──────────────────┘
                                  │
                          state update + NIS
                                  │
                       ┌──────────┴──────────┐
                       ▼                     ▼
              ┌──────────────┐       ┌──────────────┐
              │  anomaly.py  │       │  SQLite: states │
              │  (NIS test)  │       └──────────────┘
              └──────────────┘
                       │
                  alert event
                       │
                       ▼
              ┌──────────────────────────┐
              │  FastAPI WebSocket /ws/live │
              └──────────────────────────┘
                       │ JSON stream
                       ▼
              ┌──────────────────────────┐
              │  Browser                 │
              │  ├── globe.js  (CesiumJS) │
              │  ├── residuals.js  (D3)   │
              │  └── alerts.js            │
              └──────────────────────────┘
```

5. Deployment (POC)

The POC runs entirely on a single developer machine or a small cloud virtual machine (2 vCPU, 4 GB Random Access Memory (RAM) is sufficient for 50 objects at 30-minute update intervals).

Startup sequence:

```
# 1. Start backend
cd backend
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 2. Serve frontend (any static server)
cd frontend
python -m http.server 3000

# 3. For demo: pre-load TLE cache
python ../scripts/replay.py --hours 72

# 4. For demo: inject maneuver event
python ../scripts/seed_maneuver.py --object 25544 --delta-v 0.5
```

Environment variables required:

```
SPACETRACK_USER=<email>
SPACETRACK_PASS=<password>
CESIUM_ION_TOKEN=<token>
```

6. Scalability path (post-POC)

The POC architecture is deliberately simple. Each component maps to a production replacement:

POC component	Production equivalent
SQLite catalog	TimescaleDB or InfluxDB for time-series state history
Single-process FastAPI	Kubernetes microservices, one per component
Hand-tuned `Q` matrix	Adaptive process noise (SAGE/Holt estimator)
SGP4 propagator	High-fidelity numerical integrator (RK4 + full force model)
Space-Track.org only	Multi-source sensor fusion (Space Fence, commercial radars, optical)
CDN frontend	React + WebGL custom renderer for large catalog (10k+ objects)
No auth	OAuth2 + role-based access (operator vs. analyst vs. viewer)

7. Key design decisions

UKF over EKF: The measurement-to-state mapping via SGP4 is sufficiently nonlinear that the EKF's linearization error is meaningful over typical inter-update intervals (30 minutes). The UKF's sigma-point approach handles this without requiring Jacobian computation.

TLE as observation proxy: For the POC, each new TLE publication from Space-Track is treated as a noisy observation. This is a deliberate simulation of the sensor-to-catalog pipeline, not the pipeline itself. The architecture document must be explicit about this for technical reviewers.

Monorepo for POC: Backend and frontend share a single repository to minimize friction during rapid development. Production deployment would separate these into independent services.

No message queue for POC: The event bus between components is in-process Python. Adding Kafka or NATS between ingest and the Kalman layer is a named post-POC task (see requirements doc).

---

8. Security and compliance notes

- All Space-Track credentials in environment variables, never committed
- CesiumJS Ion token scoped to the specific asset set — rotate before any public presentation
- No classified data in this system. All data sources are unclassified, publicly releasable
- International Traffic in Arms Regulations (ITAR) note: Space-Track data export is controlled. Do not automate redistribution of raw TLE data to third parties without reviewing Space-Track terms
- For any Department of Defense (DoD) demo environment: confirm network connectivity to Space-Track.org is permitted, or pre-cache TLEs on an air-gap-compatible device